E-Commerce Web Application

Full Stack Project Documentation

Introduction

The E-Commerce Web Application is a full-stack web application developed using MERN stack technologies.
The application provides an online shopping platform where users can browse products,
add products to cart, place orders, and complete checkout operations successfully.

The project includes:
• User Authentication
• Product Management
• Shopping Cart Functionality
• Checkout System
• Role-Based Access Control
• Database Integration

Objective

The main objective of this project is to develop an online shopping platform with product and order management features.

Users can:
• Register and Login
• Browse Products
• Add Products to Cart
• Checkout Products
• Place Orders

Admin users can:
• Add Products

• Update Products
• Delete Products
• Manage Orders

Technologies Used

Frontend:
• React.js
• HTML
• CSS
• Bootstrap

Backend:
• Node.js
• Express.js

Database:
• MongoDB

Authentication:
• JWT Authentication

Key Features

• Product Catalog
• Add to Cart Functionality
• Checkout Functionality
• User Login System
• Role-Based Access Control
• Product Management APIs
• Order Management APIs
• Responsive User Interface
• Database Integration

Project Folder Structure

```
ecommerce-app/

■■■ client/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■   ■■■ Navbar.js
■   ■   ■   ■■■ ProductCard.js
■   ■   ■   ■■■ Cart.js
■   ■   ■   ■■■ Checkout.js
■   ■   ■■■ pages/
■   ■   ■   ■■■ Login.js
■   ■   ■   ■■■ Register.js
■   ■   ■   ■■■ Home.js
■   ■   ■   ■■■ Products.js
■   ■   ■   ■■■ Orders.js
■   ■   ■■■ App.js
■   ■   ■■■ api.js
■   ■   ■■■ index.js
■
■■■ server/
■   ■■■ models/
■   ■   ■■■ User.js
■   ■   ■■■ Product.js
■   ■   ■■■ Order.js
■   ■■■ routes/
■   ■   ■■■ authRoutes.js
■   ■   ■■■ productRoutes.js
■   ■   ■■■ orderRoutes.js
■   ■■■ server.js
■   ■■■ package.json
```

Frontend Setup

```
npx create-react-app client

cd client

npm install axios react-router-dom bootstrap
```

index.js

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

api.js

```
import axios from "axios";

export default axios.create({
  baseURL: "http://localhost:5000/api"
});
```

App.js

```
import React from "react";
```

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Home from "./pages/Home";
import Login from "./pages/Login";
import Register from "./pages/Register";
import Products from "./pages/Products";
import Orders from "./pages/Orders";

function App() {

  return (

    <BrowserRouter>

      <Routes>

        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/products" element={<Products />} />
        <Route path="/orders" element={<Orders />} />

      </Routes>

    </BrowserRouter>

  );
}

export default App;
```

Login.js

```
import React, { useState } from "react";

import api from "../api";

function Login() {

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const loginUser = async () => {

    const res = await api.post("/auth/login", {
      email,
      password
    });

    localStorage.setItem("token", res.data.token);

    alert("Login Successful");
  };

  return (

    <div>

      <h2>Login</h2>

      <input
        type="email"
        placeholder="Enter Email"
        onChange={(e) => setEmail(e.target.value)}
      />
```

```
      <input
        type="password"
        placeholder="Enter Password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <button onClick={loginUser}>
        Login
      </button>

    </div>

  );
}

export default Login;
```

Register.js

```
import React, { useState } from "react";

import api from "../api";

function Register() {

  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const registerUser = async () => {

    await api.post("/auth/register", {
      name,
      email,
      password
    });

    alert("Registration Successful");
  };

  return (

    <div>

      <h2>Register</h2>

      <input
        type="text"
        placeholder="Enter Name"
        onChange={(e) => setName(e.target.value)}
      />

      <input
        type="email"
        placeholder="Enter Email"
        onChange={(e) => setEmail(e.target.value)}
      />

      <input
        type="password"
        placeholder="Enter Password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <button onClick={registerUser}>
```

```
        Register
      </button>

    </div>

  );
}

export default Register;
```

Products.js

```
import React, { useEffect, useState } from "react";

import api from "../api";

function Products() {

  const [products, setProducts] = useState([]);
  const [cart, setCart] = useState([]);

  useEffect(() => {
    fetchProducts();
  }, []);

  const fetchProducts = async () => {

    const res = await api.get("/products");

    setProducts(res.data);
  };

  const addToCart = (product) => {

    setCart([...cart, product]);

    alert("Product Added to Cart");
  };

  return (

    <div>

      <h2>Products</h2>

      {
        products.map((product) => (

          <div key={product._id}>

            <h4>{product.name}</h4>

            <p>{product.price}</p>

            <button onClick={() => addToCart(product)}>
              Add to Cart
            </button>

          </div>

        ))
      }

    </div>
  );
```

```
}

export default Products;
```

Checkout.js

```
import React from "react";

function Checkout() {

  const checkoutHandler = () => {

    alert("Order Placed Successfully");

  };

  return (

    <div>

      <h2>Checkout Page</h2>

      <button onClick={checkoutHandler}>
        Place Order
      </button>

    </div>

  );
}

export default Checkout;
```

Backend Setup

```
npm init -y

npm install express mongoose cors dotenv bcryptjs jsonwebtoken nodemon
```

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

require("dotenv").config();

const authRoutes = require("./routes/authRoutes");
const productRoutes = require("./routes/productRoutes");
const orderRoutes = require("./routes/orderRoutes");

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)

.then(() => console.log("MongoDB Connected"))
.catch((err) => console.log(err));
```

```
app.use("/api/auth", authRoutes);
app.use("/api/products", productRoutes);
app.use("/api/orders", orderRoutes);

const PORT = 5000;

app.listen(PORT, () => {

  console.log(`Server running on port ${PORT}`);

});
```

User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({

  name: String,
  email: String,
  password: String,

  role: {
    type: String,
    default: "user"
  }

});

module.exports = mongoose.model("User", userSchema);
```

Product.js

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({

  name: String,
  price: Number,
  description: String,
  image: String

});

module.exports = mongoose.model("Product", productSchema);
```

Order.js

```
const mongoose = require("mongoose");

const orderSchema = new mongoose.Schema({

  userId: String,
  products: Array,
  totalPrice: Number,

  orderStatus: {
    type: String,
    default: "Pending"
  }
```

```
});

module.exports = mongoose.model("Order", orderSchema);
```

authRoutes.js

```
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

const User = require("../models/User");

const router = express.Router();

router.post("/register", async (req, res) => {

  const { name, email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  const user = new User({
    name,
    email,
    password: hashedPassword
  });

  await user.save();

  res.json({
    message: "User Registered Successfully"
  });

});

router.post("/login", async (req, res) => {

  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {
    return res.json({
      message: "User Not Found"
    });
  }

  const isMatch = await bcrypt.compare(password, user.password);

  if (!isMatch) {
    return res.json({
      message: "Invalid Password"
    });
  }

  const token = jwt.sign(
    { id: user._id },
    "secretkey"
  );

  res.json({ token });

});

module.exports = router;
```

productRoutes.js

```
const express = require("express");

const Product = require("../models/Product");

const router = express.Router();

router.get("/", async (req, res) => {

  const products = await Product.find();

  res.json(products);

});

router.post("/", async (req, res) => {

  const product = new Product(req.body);

  await product.save();

  res.json(product);

});

router.put("/:id", async (req, res) => {

  const updatedProduct = await Product.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );

  res.json(updatedProduct);

});

router.delete("/:id", async (req, res) => {

  await Product.findByIdAndDelete(req.params.id);

  res.json({
    message: "Product Deleted Successfully"
  });

});

module.exports = router;
```

orderRoutes.js

```
const express = require("express");

const Order = require("../models/Order");

const router = express.Router();

router.post("/", async (req, res) => {

  const order = new Order(req.body);

  await order.save();
  res.json(order);
```

```
});

router.get("/", async (req, res) => {

  const orders = await Order.find();

  res.json(orders);

});

module.exports = router;
```

Environment Variables

```
MONGO_URI=your_mongodb_connection

JWT_SECRET=secretkey
```

Run Backend Server

```
cd server

npm install

npm run dev
```

Run Frontend Application

```
cd client

npm install

npm start
```

Conclusion

The E-Commerce Web Application is a useful MERN stack project developed using React.js, Node.js, Express.js, and MongoDB.

This project helps in understanding:

• Frontend and Backend Integration
• Product Management Systems
• Shopping Cart Functionality
• Authentication Systems
• REST API Development
• Database Integration